

Weekly Progress Report

Name: **Mythreyan R.B**

Domain: **Data Science and Machine Learning**

Date of submission: **29.08.2026**

Week Ending: 02

I. Overview:

This week, the primary focus transitioned from initial baseline exploration to designing and deploying an interactive web application for the project Quality Prediction in a Mining Process. Efforts centered on leveraging Streamlit to build a user-facing dashboard that calculates and visualizes mining quality predictions based on key process inputs.

II. Achievements:

1. Streamlit Application Development

Designed and built an interactive web interface using Streamlit deployed locally via Jupyter Lab and Anaconda Navigator .Developed input fields (sliders and numerical inputs) for critical engineering features: pH levels, slurry intensity, and chemical reagent dosing rates. Created reactive dashboard elements that dynamically display predicted mining quality variations upon user input adjustments.

2. Model Feature Integration

Restricted the model inference pipeline to focus tightly on the most impactful features identified during exploratory data analysis: pH, slurry intensity, and reagent dose. Standardized the app's internal pipeline to correctly format user interface inputs before passing them to the trained machine learning backend.

3. Data Visualization & UI Enhancements

Implemented real-time plotting using Matplotlib and Seaborn directly within the Streamlit UI to track historical vs. predicted quality deviations. Streamlined the web page layout for industrial operators, ensuring quick visibility of critical quality metrics.

III. Challenges:

1. Session State and UI Lag

Encountered performance bottlenecks and widget state loss when dynamically refreshing charts based on concurrent changes in slurry intensity and dosing sliders. Troubleshooting optimal uses of [*st.cache_resource*](#) and [*st.session_state*](#) to prevent unnecessary model re-loading during user interactions.

2. Out-of-Bounds Input Handling

Faced difficulties mapping highly skewed historical operational ranges (caused by sensor anomalies and plant downtime) into realistic UI bounding constraints for the sliders.

IV. Learning Resources:

1. Streamlit Core Documentation

Utilized the official Streamlit layout, widget, and caching guides to resolve session reactivity issues. Reviewed open-source industrial dashboard templates on GitHub for UI design best practices.

2. Advanced Feature Engineering & Deployment

Explored advanced tutorials on embedding pre-trained Scikit-Learn models ([*pickle/joblib*](#) file tracking) into runtime application architectures.

V. Next Week's Goals:

1. Simple Application Construction

Build a functional, lightweight user interface using Streamlit to serve as a mining quality calculator. Integrate input controllers (such as sliders or numerical boxes) for critical operational variables: pH, slurry intensity, and reagent dose.

2. Logic and Validation Pipeline

Implement the core calculation logic inside the app to process input parameters and compute quality outputs instantly. Set up a chronological validation loop to benchmark user-input calculations against baseline industrial parameters..

VI. Additional Comments:

- Preliminary performance tests confirm that tracking user modifications specifically to slurry intensity and chemical reagent dosing rates inside the app provides the most immediate, interpretable feedback regarding final quality metrics..